

We just came up with two really general and potentially useful combinators based purely on the abstract interface of `Functor`, and we can reuse them for any type that allows an implementation of `map`.

11.1.1 Functor laws

Whenever we create an abstraction like `Functor`, we should consider not only what abstract methods it should have, but which *laws* we expect to hold for the implementations. The laws you stipulate for an abstraction are entirely up to you,³ and of course Scala won't enforce any of these laws. But laws are important for two reasons:

- Laws help an interface form a new semantic level whose algebra may be reasoned about *independently* of the instances. For example, when we take the product of a `Monoid[A]` and a `Monoid[B]` to form a `Monoid[(A, B)]`, the monoid laws let us conclude that the “fused” monoid operation is also associative. We don't need to know anything about `A` and `B` to conclude this.
- More concretely, we often rely on laws when writing various combinators derived from the functions of some abstract interface like `Functor`. We'll see examples of this later.

For `Functor`, we'll stipulate the familiar law we first introduced in chapter 7 for our `Par` data type:⁴

```
map(x) (a => a) == x
```

In other words, mapping over a structure `x` with the identity function should itself be an identity. This law is quite natural, and we noticed later in part 2 that this law was satisfied by the `map` functions of other types besides `Par`. This law (and its corollaries given by parametricity) capture the requirement that `map(x)` “preserves the structure” of `x`. Implementations satisfying this law are restricted from doing strange things like throwing exceptions, removing the first element of a `List`, converting a `Some` to `None`, and so on. Only the elements of the structure are modified by `map`; the shape or structure itself is left intact. Note that this law holds for `List`, `Option`, `Par`, `Gen`, and most other data types that define `map`!

To give a concrete example of this preservation of structure, we can consider `distribute` and `codistribute`, defined earlier. Here are their signatures again:

```
def distribute[A,B](fab: F[(A, B)]): (F[A], F[B])
```

```
def codistribute[A,B](e: Either[F[A], F[B]]): F[Either[A, B]]
```

Since we know nothing about `F` other than that it's a functor, the law assures us that the returned values will have the same shape as the arguments. If the input to `distribute` is a list of pairs, the returned pair of lists will be of the same length as the

³ Though if you're going to borrow the name of some existing mathematical abstraction like functor or monoid, we recommend using the laws already specified by mathematics.

⁴ This law also comes from the mathematical definition of functor.